

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
КАФЕДРА АВТОМАТИЗАЦІЇ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

ЗВІТ

ЗВІТ З ЛАБОРАТОРНИХ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
**«СУЧАСНІ ТЕХНОЛОГІЇ КОНТЕЙНЕРИЗАЦІЇ, РОЗГОРТАННЯ ТА
ПОБУДОВИ МАСШТАБОВАНИХ ПРОГРАМНИХ СИСТЕМ»**

Виконав студент групи КН-23-1

Полинько Ігор Миколайович

Перевірив: викладач Юдіна А. Л.

Кременчук 2026

ЛАБОРАТОРНА РОБОТА № 1

Тема: Установлення Docker Desktop та основи роботи з Docker

Мета: ознайомитися з архітектурою Docker, підготувати Windows-середовище для запуску Linux-контейнерів, установити та налаштувати Docker Desktop, навчитися знаходити й завантажувати образи, створювати контейнери, переглядати їхній стан і журнали, повторно запускати та видаляти контейнери й образи.

Порядок виконання роботи:

1. Перевірити версію Windows і стан апаратної віртуалізації.
2. Установити або перевірити WSL 2 без обов'язкового встановлення Ubuntu-дистрибутива.
3. Завантажити й установити Docker Desktop.
4. Налаштувати WSL 2 backend і режим Linux containers.
5. Перевірити версії Docker CLI, Docker Engine і Docker Compose.
6. Створити обліковий запис Docker Hub та виконати вхід.
7. Знайти й завантажити образ hello-world.
8. Запустити перший контейнер і пояснити отримане повідомлення.
9. Переглянути локальні образи, контейнери та журнал контейнера.
10. Створити контейнер із власним ім'ям і повторно запустити його.
11. Видалити створені контейнери й образ.
12. Виконати індивідуальне завдання.
13. Оформити звіт та відповісти на контрольні питання.

Індивідуальне завдання:

Виконайте варіант, визначений викладачем. Для свого образу потрібно: знайти

- офіційну сторінку або сторінку перевіреного видавця в Docker Hub;
- виконати pull;
- визначити розмір, архітектуру й ОС;
- запустити задану команду;
- створити іменованний контейнер;

- переглянути ps -a та logs;
- повторно запустити контейнер із приєднанням до виведення;
- видалити контейнер і образ.

Варіант 15. Node.js

PowerShell

```
docker pull node:alpine
```

```
docker image inspect node:alpine
```

```
docker run --name variant-node node:alpine node -e
```

```
"console.log(process.version); console.log(process.platform)"
```

```
docker logs variant-node
```

```
docker start -a variant-node
```

1. Визначте версію Node.js.
2. Визначте платформу процесу.
3. Поясніть роль Alpine у цьому образі.

Хід роботи:

1. Версія Windows:

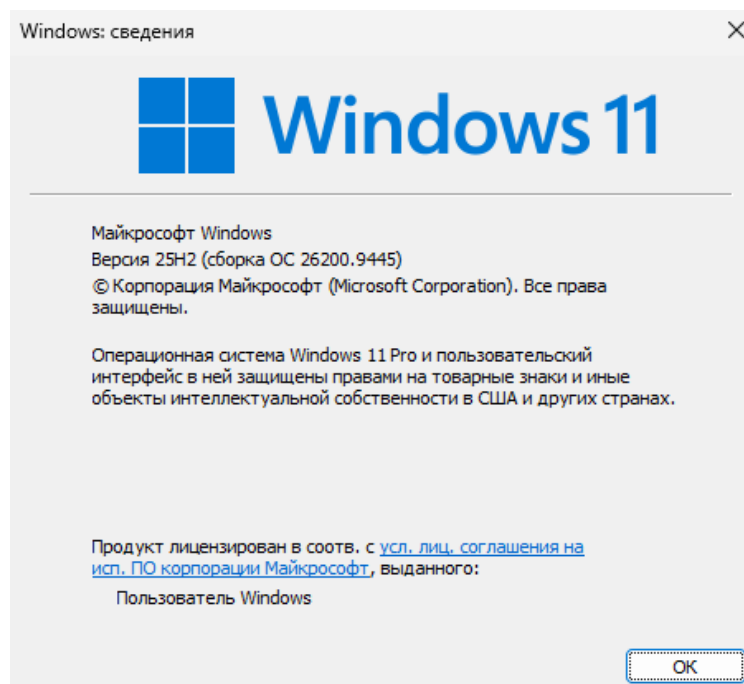


Рисунок 1.1 — Редакція, версія та номер збірки Windows 11

2. Апаратна віртуалізація:

Базовая скорость:	2,00 ГГц
Сокетов:	1
Ядра:	8
Логических процессоров:	12
Виртуализация:	Включено
Кэш L1:	704 КБ
Кэш L2:	7,0 МБ
Кэш L3:	12,0 МБ

Рисунок 1.2 — Ввімкнений параметр віртуалізації

3-6. Наявність WSL:

```
PS C:\WINDOWS\system32> wsl --version
Версия WSL: 2.7.14.0
Версия ядра: 6.18.33.2-2
Версия WSLg: 1.0.73.2
Версия MSRDC: 1.2.7214
Версия Direct3D: 1.611.1-81528511
Версия DXCore: 10.0.26100.1-240331-1435.ge-release
Версия Windows: 10.0.26200.9457
PS C:\WINDOWS\system32> wsl --status
Версия по умолчанию: 2
```

Рисунок 1.3 — Версія WSL, версія ядра і основні
Параметри

7. Установка Docker Desktop:

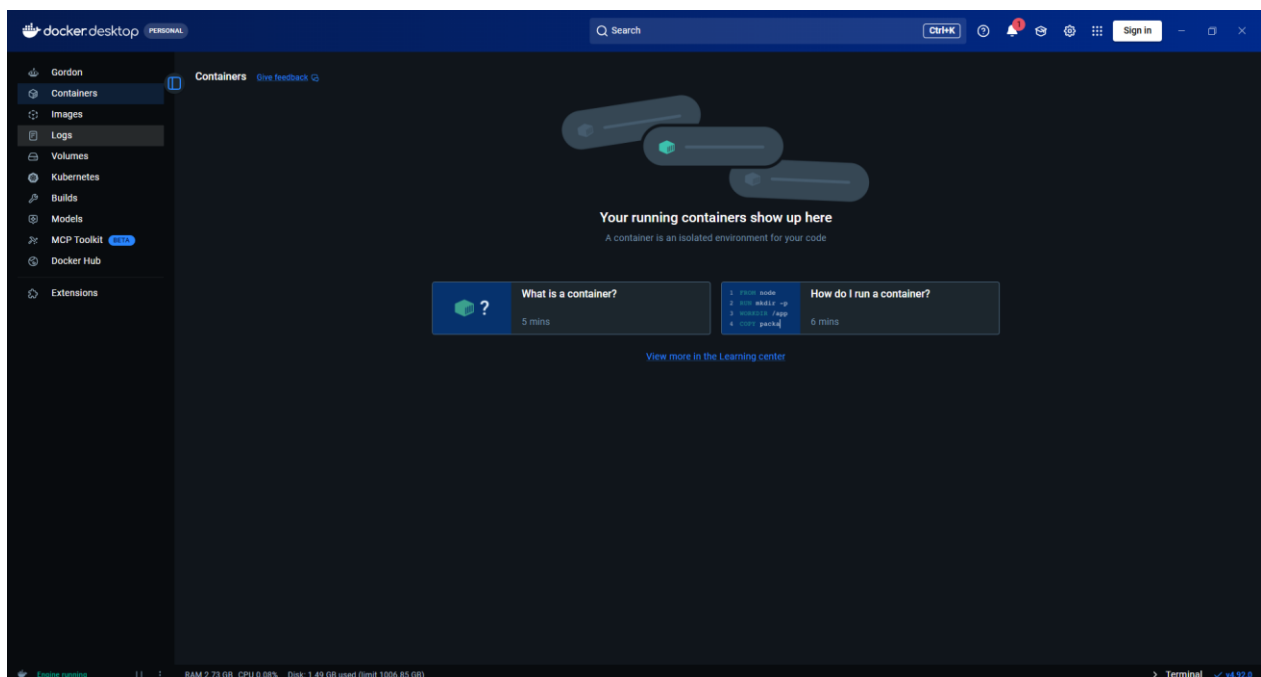


Рисунок 1.4 — Docker Desktop

8. Основні налаштування Docker Desktop:

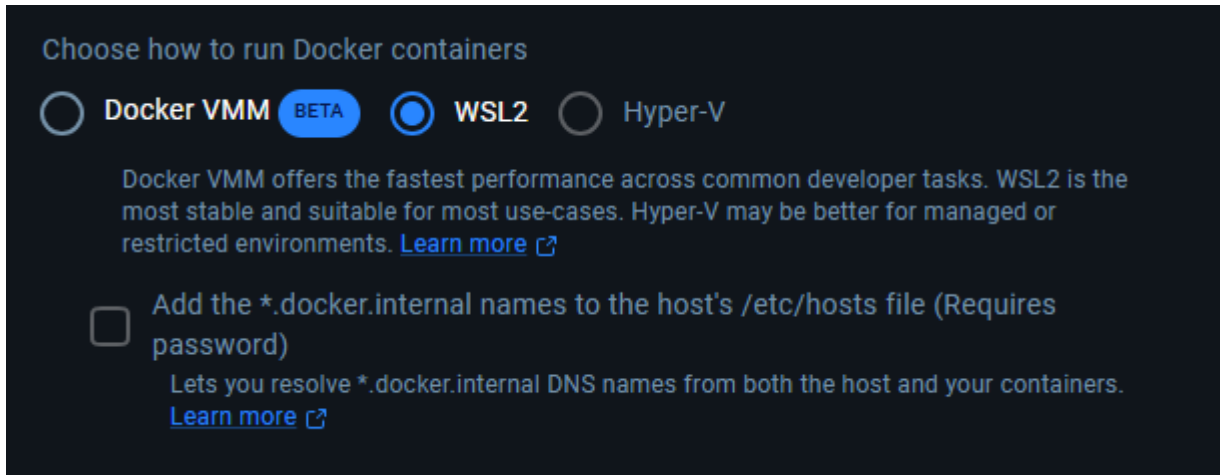


Рисунок 1.5 — WSL 2 based engine використовується

9. Перевірка Docker CLI, Engine і Compose:

```
PS C:\WINDOWS\system32> docker --version
Docker version 29.8.0, build 88096ef
PS C:\WINDOWS\system32> docker version
Client:
 Version:           29.8.0
 API version:       1.56
 Go version:        go1.26.8
 Git commit:        88096ef
 Built:             Thu Sep  3 21:53:38 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.92.0 (240144)
 Engine:
  Version:           29.8.0
  API version:       1.56 (minimum version 1.40)
  Go version:        go1.26.8
  Git commit:        3ce5872
  Built:             Thu Sep  3 21:51:20 2026
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           v2.3.5
  GitCommit:         1294c24a7da8e5a793ed378161673abe94118892
 runc:
  Version:           1.5.1
  GitCommit:         v1.5.1-0-g8f2685a4
 docker-init:
  Version:           0.19.0
  GitCommit:         de40ad0
PS C:\WINDOWS\system32> docker info --format '{{.OSType}}'
linux
PS C:\WINDOWS\system32> docker compose version
Docker Compose version v5.5.1
```

Рисунок 1.6 — Docker CLI, Engine і Compose

10. Обліковий запис Docker Hub:

```
PS C:\WINDOWS\system32> docker login
Authenticating with existing credentials... [Username: ihorpolyenko]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
```

Рисунок 1.7 — Docker Login

11. Образ hello-world:

```
PS C:\WINDOWS\system32> docker search hello-world
```

NAME	DESCRIPTION	STARS	OFFICIAL
hello-world	Hello World! (an example of minimal Dockeriz...	2593	[OK]
rancher/hello-world	This container image is no longer maintained...	6	
okteto/hello-world		0	
atlassian/hello-world		3	
goharbor/hello-world		0	
tutum/hello-world	Image to test docker deployments. Has Apache...	91	
dockercloud/hello-world	Hello World!	20	
crccheck/hello-world	Hello World web server in under 2.5 MB	26	
koudaiiii/hello-world		0	
prajwalendra/hello-world		0	
ppc64le/hello-world	Hello World! (an example of minimal Dockeriz...	2	
tsepotesting123/hello-world		0	
infrastructureascode/hello-world	A tiny "Hello World" web server with a healt...	1	
kevindockercompany/hello-world		0	
arm32v7/hello-world	Hello World! (an example of minimal Dockeriz...	3	
arm64v8/hello-world	Hello World! (an example of minimal Dockeriz...	3	
datawire/hello-world	Hello World! Simple Hello World implementati...	1	
cloudflare/hello-world	A simple example application which can be ru...	0	
uniplaces/hello-world		0	
twistlocktest/hello-world		0	
wjimenez5271/hello-world		0	
danfengliu/hello-world		0	
lbadger/hello-world		0	
ansibleplaybookbundle/hello-world	Simple containerized application that tests ...	0	
kousik93/hello-world		0	

Рисунок 1.8 — Пошук hello-world

12. Образ hello-world:

```
PS C:\WINDOWS\system32> docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Image is up to date for hello-world:latest
docker.io/library/hello-world:latest
PS C:\WINDOWS\system32> docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	5e2309035332	25.9kB	9.49kB	

Info → U In Use

Рисунок 1.9 — Пошук ідентифікатора образу, архітектури, операційної системи та команди запуску

```

PS C:\WINDOWS\system32> docker image inspect hello-world
[
  {
    "Id": "sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8",
    "RepoTags": [
      "hello-world:latest"
    ],
    "RepoDigests": [
      "hello-world@sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-03-23T21:33:59.562202219Z",
    "Config": {
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
      ],
      "Cmd": [
        "/hello"
      ],
      "WorkingDir": "/"
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 20399,
    "RootFS": {
      "Type": "layers",
      "Layers": [
        "sha256:897b3f2a7c1bc2f3d02432f7892fe31c6272c521ad4d70257df624504a3238b4"
      ]
    },
    "Metadata": {
      "LastTagTime": "2026-09-27T17:19:21.802123629Z"
    },
    "Descriptor": {
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "digest": "sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8",
      "size": 11062
    },
    "Identity": {
      "Pull": [
        {
          "Repository": "docker.io/library/hello-world"
        }
      ]
    }
  ]
}
]

```

Рисунок 1.10 — Локальне сховище образів

13. Запуск першого контейнеру:

```

PS C:\WINDOWS\system32> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Рисунок 1.11 — Контейнер hello-world

14. Перегляд контейнерів:

```
PS C:\WINDOWS\system32> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED      STATUS      PORTS     NAMES
8bb95b83adc4   hello-world  "/hello"  4 minutes ago  Exited (0)  4 minutes ago  thirsty_goldstine
PS C:\WINDOWS\system32> docker container ls -a
CONTAINER ID   IMAGE     COMMAND   CREATED      STATUS      PORTS     NAMES
8bb95b83adc4   hello-world  "/hello"  4 minutes ago  Exited (0)  4 minutes ago  thirsty_goldstine
```

Рисунок 1.12 — Перегляд контейнерів

15. Створення контейнеру із заданим ім'ям:

```
PS C:\WINDOWS\system32> docker run --name student-hello hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED      STATUS      PORTS     NAMES
95f2ea1cec41   hello-world  "/hello"  7 seconds ago  Exited (0)  6 seconds ago  student-hello
8bb95b83adc4   hello-world  "/hello"  7 minutes ago  Exited (0)  7 minutes ago  thirsty_goldstine
```

Рисунок 1.13 — Контейнер student-hello

16. Перегляд журналу контейнера:

```
PS C:\WINDOWS\system32> docker logs student-hello
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Рисунок 1.14 — Журнал контейнера

17. Повторний запуск створеного контейнера:

```
PS C:\WINDOWS\system32> docker start student-hello
student-hello
PS C:\WINDOWS\system32> docker start -a student-hello

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Рисунок 1.15 — Повторний запуск контейнера student-hello

18. Порівняння docker run і docker start:

```
PS C:\WINDOWS\system32> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\WINDOWS\system32> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS          NAMES
b866e259a2b3   hello-world    "/hello"                7 seconds ago   Exited (0) 6 seconds ago           loving_montalcini
ddb681ebf063   hello-world    "/hello"                16 seconds ago   Exited (0) 15 seconds ago           stoic_dirac
95f2ea1ce41    hello-world    "/hello"                9 minutes ago   Exited (0) 5 minutes ago           student-hello
8bb95b83adc4   hello-world    "/hello"                16 minutes ago   Exited (0) 16 minutes ago           thirsty_goldstine
PS C:\WINDOWS\system32>
```

Рисунок 1.16 — Порівняння docker run і docker start

19. Видалення створених контейнерів:

```
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
b866e259a2b3   hello-world    "/hello"                2 minutes ago Exited (0)    2 minutes ago loving_montalcini
ddb681ebf063   hello-world    "/hello"                2 minutes ago Exited (0)    2 minutes ago stoic_dirac
95f2ea1cec41   hello-world    "/hello"                11 minutes ago Exited (0)    7 minutes ago student-hello
8bb95b83adc4   hello-world    "/hello"                18 minutes ago Exited (0)    18 minutes ago thirsty_goldstine
PS C:\WINDOWS\system32> docker rm student-hello
student-hello
PS C:\WINDOWS\system32> docker rm <b866e259a2b3>
строка:1 знак:11
+ docker rm <b866e259a2b3>
+ ~
Оператор "<" зарезервирован для использования в будущем.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : RedirectionNotSupported

PS C:\WINDOWS\system32> docker rm b866e259a2b3
b866e259a2b3
PS C:\WINDOWS\system32> docker rm ddb681ebf063
ddb681ebf063
PS C:\WINDOWS\system32> docker rm 8bb95b83adc4
8bb95b83adc4
```

Рисунок 1.17 — Видалення створених контейнерів

20. Видалення образу і перевірка очищення:

```
PS C:\WINDOWS\system32> docker image rm hello-world
Untagged: hello-world:latest
Deleted: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
PS C:\WINDOWS\system32> docker image ls
Info → U In Use

IMAGE ID          DISK USAGE  CONTENT SIZE  EXTRA
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
```

Рисунок 1.17 — Видалення образу і перевірка очищення

Індивідуальне завдання:

```
PS C:\WINDOWS\system32> docker pull node:alpine
alpine: Pulling from library/node
e77339b8f20b: Pull complete
e2de96513ba9: Pull complete
b999169ad112: Pull complete
d9f017433cab: Download complete
eb51d8194c35: Download complete
Digest: sha256:0b36e8c136b94cd4fcf02188228e76c31ad5872eef3fec8cbd2eee500cfd9e80
Status: Downloaded newer image for node:alpine
docker.io/library/node:alpine
```

Рисунок 1.18 — Виконання pull

```

PS C:\WINDOWS\system32> docker image inspect node:alpine
[
  {
    "Id": "sha256:0b36e8c136b94cd4fcf02188228e76c31ad5872eef3fec8cbd2eee500cfd9e80",
    "RepoTags": [
      "node:alpine"
    ],
    "RepoDigests": [
      "node@sha256:0b36e8c136b94cd4fcf02188228e76c31ad5872eef3fec8cbd2eee500cfd9e80"
    ],
    "Comment": "buildkit.dockerfile.v0",
    "Created": "2026-09-22T18:51:35.843212472Z",
    "Config": {
      "Env": [
        "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
        "NODE_VERSION=26.10.0"
      ],
      "Entrypoint": [
        "docker-entrypoint.sh"
      ],
      "Cmd": [
        "node"
      ],
      "WorkingDir": "/"
    },
    "Architecture": "amd64",
    "Os": "linux",
    "Size": 258303821,
    "RootFS": {
      "Type": "layers",
      "Layers": [
        "sha256:74d97c428c51a828f9051a7a40a53ff1fc99e54fc30323ce36760701b0b7f711",
        "sha256:2dd79dc60bb272930cebd0998c26ed644009cfde8b732034e5638b2f21a95d4f",
        "sha256:c8a74b39fa8f6230e88854b0494f38898311ac1bcfa906afaca5b75fa4ebce9d"
      ]
    },
    "Metadata": {
      "LastTagTime": "2026-09-27T17:53:06.560228633Z"
    },
    "Descriptor": {
      "mediaType": "application/vnd.oci.image.index.v1+json",
      "digest": "sha256:0b36e8c136b94cd4fcf02188228e76c31ad5872eef3fec8cbd2eee500cfd9e80",
      "size": 2613
    },
    "Identity": {
      "Pull": [
        {
          "Repository": "docker.io/library/node"
        }
      ]
    }
  }
]

```

Рисунок 1.19 — Характеристики образу

```

PS C:\WINDOWS\system32> docker run --name variant-node node:alpine node -e "console.log(process.version); console.log(process.platform)"
v26.10.0
linux
PS C:\WINDOWS\system32> docker logs variant-node
v26.10.0
linux
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
4933ad5ea117   node:alpine "docker-entrypoint.s..." 29 seconds ago Exited (0) 29 seconds ago           variant-node
PS C:\WINDOWS\system32> docker start -a variant-node
v26.10.0
linux
PS C:\WINDOWS\system32> docker rm variant-node
variant-node
PS C:\WINDOWS\system32> docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
PS C:\WINDOWS\system32> docker image rm node:alpine
Error response from daemon: No such image: node:alpine:latest
PS C:\WINDOWS\system32> docker image rm node:alpine
Untagged: node:alpine
Deleted: sha256:0b36e8c136b94cd4fcf02188228e76c31ad5872eef3fec8cbd2eee500cfd9e80
PS C:\WINDOWS\system32> docker image ls

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
PS C:\WINDOWS\system32> _				

Info → In Use

Рисунок 1.20 — Запуск заданої команду, створення іменованого контейнера, перегляд ps -a та logs, повторний запуск контейнера із приєднанням до виведення, видалення контейнера і образу

Висновок:

Для виконання індивідуального завдання було використано офіційний образ node:alpine.

У результаті виконання команди `docker image inspect node:alpine` встановлено:

- операційна система образу – linux;
- архітектура – amd64;
- розмір образу – 258 303 821 байт (приблизно 246,3 MiB).

Після запуску контейнера командою

```
docker run --name variant-node node:alpine node -e "console.log(process.version);  
console.log(process.platform)"
```

отримано:

- версія Node.js – v26.10.0;
- платформа процесу – linux.

Контейнер завершив роботу зі статусом Exited (0), що означає успішне виконання команди без помилки.

Alpine Linux у цьому образі використовується як легкий базовий дистрибутив Linux. Його застосування дозволяє зменшити розмір образу та кількість ресурсів, необхідних для запуску контейнера, порівняно з образами Node.js на базі повнорозмірних дистрибутивів Linux.

Контрольні питання:

1. Що таке контейнеризація?

Контейнеризація – це спосіб запуску програм у ізольованих контейнерах разом з необхідними залежностями та налаштуваннями.

2. Для чого призначений Docker Desktop?

Docker Desktop – це застосунок для Windows, який забезпечує встановлення, налаштування та керування Docker, контейнерами й образами через графічний інтерфейс та командний рядок.

3. Яку роль виконує Docker Engine?

Docker Engine – це основний компонент Docker, який створює, запускає та керує контейнерами, образами, мережами та іншими ресурсами Docker.

4. Чим Docker CLI відрізняється від Docker Engine?

Docker CLI – це командний інтерфейс, через який користувач надсилає команди Docker. Docker Engine приймає ці команди та безпосередньо виконує необхідні операції.

5. Як перевірити поточний тип контейнерів?

Поточний тип контейнерів можна перевірити командою:

```
docker info --format '{{.OSType}}'
```

Для Linux-контейнерів команда повертає linux.

6. Що таке Docker-образ?

Docker-образ – це незмінний шаблон, який містить файлову систему, програму, залежності та налаштування, необхідні для створення контейнера.

7. Що таке контейнер?

Контейнер – це запущений або зупинений екземпляр Docker-образу, який працює ізольовано від інших контейнерів.

8. Чим образ відрізняється від контейнера?

Образ є шаблоном для створення контейнерів, а контейнер є конкретним екземпляром цього образу, який можна запускати та зупиняти.

9. Що таке реєстр контейнерних образів?

Реєстр контейнерних образів – це сервер або сервіс для зберігання, поширення та завантаження Docker-образів. Прикладом є Docker Hub.

10. Що таке репозиторій і тег образу?

Репозиторій – це сховище образів певного програмного продукту. Тег – це позначка, яка визначає конкретну версію або варіант образу, наприклад node:alpine.

11. Що означає тег latest?

latest – це стандартний тег, який зазвичай позначає версію образу, яку видавець вважає актуальною за замовчуванням. Він не обов'язково означає найновішу версію програми.

12. Для чого використовується docker pull?

`docker pull` використовується для завантаження Docker-образу з реєстру, наприклад Docker Hub, на локальний комп'ютер.

13. Які дії послідовно виконує `docker run`?

`docker run` перевіряє наявність образу локально, за потреби завантажує його, створює контейнер і запускає його.

14. Чим `docker run` відрізняється від `docker start`?

`docker run` створює новий контейнер та запускає його. `docker start` запускає вже існуючий зупинений контейнер.

15. Чому контейнер `hello-world` одразу переходить у стан `Exited`?

Тому що `hello-world` призначений лише для виведення тестового повідомлення. Після виконання своєї єдиної команди процес завершується, тому контейнер зупиняється.

16. Що означає код завершення 0?

Код завершення 0 означає, що процес завершився успішно та без помилок.

17. Як призначити контейнеру власне ім'я?

Ім'я контейнеру призначається за допомогою параметра `--name`, наприклад:
`docker run --name my-container hello-world`

18. Чому не можна створити два контейнери з однаковим ім'ям?

Тому що ім'я контейнера має бути унікальним у межах Docker Engine. Воно використовується для однозначної ідентифікації контейнера.

19. Чи видаляється образ після `docker rm`?

Ні. `docker rm` видаляє контейнер, але образ, на основі якого він був створений, залишається локально.

20. Чи видаляється образ із Docker Hub після `docker image rm`?

Ні. `docker image rm` видаляє локальну копію образу з комп'ютера. Образ у Docker Hub при цьому не видаляється.